

Week 2 – Session 3 – Lab 3

Lab 1: Identify Entities and Attributes from a Flat File

You are given a CSV export from an e-commerce app that mixes order, customer, and product details in each row. Identify the candidate entities and list their key attributes.

Sample CSV rows (columns):

order_id , order_date, customer_name, customer_email, product_id, product_name, quantity, price, shipping_address

Entity	Attributes
Order	Order_id,order_date,customer_id(FK)price
Customer	Customer_name,customer_email,shipping address,customer_id(PK)
Product	Product_id,product_name,,quantity,

Lab 2: Classify Relationships and Keys

Using entities from Lab 1, state relationship types and propose PK/FK for each table. Include examples where orders contain multiple products.

Scenario / Input

Entities: Customer, Order, Product, OrderItem

Entities	Relationship types	Primary Key	Foreign Key	Example
Customer	1:N	Customer_id	-	
Order	1:N	Order_id	customer_id	
Product	M:M	Product_id	-	
OrderItem	M:M	Order_item_id	Order_id,product_id	

3: Apply First Normal Form (1NF)

transform the following denormalized row into 1NF rows.

Scenario / Input

One row:

```
order_id: 1002 customer: Bob Smith products: [ {"product_id"
, "qty"
}, {"product_id"
, "qty"
} ]
```

Answer--

1NF Rows:

Order_id, customer, product_id, qty

1002, Bob Smith, 201, 1

1002, Bob Smith, 202, 3

Lab 4: Convert Repeating Groups into Rows

Given a small JSON list of orders with product arrays, flattened into OrderItem rows using Python.

Scenario / Input

```
orders = [
    {"order_id": 1003, "customer": "Ana", "items": [{"product_id": 301, "qty": 2}]},
    {"order_id": 1004, "customer": "Ben", "items": [{"product_id": 302, "qty": 1}, {"product_id": 303, "qty": 4}]}
]
```

Answer --

Python

```
orders=[{
    "Order_id":1003,
    "customer":"Ana",
    "items":[{"product_id":301,"qty":2}]
},
{
    "Order_id":1004,
    "customer":"Ben",
    "items":[{"product_id":302,"qty":4},{"product_id":303,"qty":4}]
}
]

order_items=[]
for order in orders:
    for item in order["items"]:
        order_items.append({
            "Order_id":order["Order_id"],
            "customer":order["customer"],
            "product_id":item["product_id"],
            "qty":item["qty"]
        })

print(order_items)

[{'Order_id': 1003, 'customer': 'Ana', 'product_id': 301, 'qty': 2}, {'Order_id': 1004, 'customer': 'Ben', 'product_id': 302, 'qty': 4}, {'Order_id': 1004, 'customer': 'Ben', 'product_id': 303, 'qty': 4}]
```

Lab 5: Apply Second Normal Form (2NF)

You have a table OrderItem(order_id, product_id, product_name, product_price, qty). Identify partial dependencies and split tables to reach 2NF.

Scenario / Input

Columns and sample rows:

order_id	product_id	product_name	product_price	qty
----------	------------	--------------	---------------	-----

1005	401	Gizmo	9.99	2
------	-----	-------	------	---

1005	402	Doodad	5.00	1
------	-----	--------	------	---

Answer--

Product(product_id PK,product_name,product_price)

OrderItem(order_id PK,product_id FK,qty)

Example:

401,Gizmo,2

402,Doodad,1

1005,401,2

1005,402,1

Lab 6: Apply Third Normal Form (3NF)

Given Customer (customer_id, name, city, zip, state_name), identify transitive dependencies and suggest table splits to reach 3NF.

Scenario / Input

state_name depends on zip via zip->state mapping.

Answer---

Customer (customer_id PK,name,city)

Details (zip PK,customer_id FK, city,state_name)

State_name functionally depends on Zip and indirectly depends on customer_id which is transitive dependencies.

Lab 7: Design an OLTP Normalized Schema (Orders)

Design a normalized OLTP schema for customers, orders, order_items, and products. Provide CREATE TABLE statements in SQL (simplified) with PK and FK constraints.

Scenario / Input

Assume typical e-commerce needs: enforce data integrity, avoid redundancy.

```
1 CREATE TABLE customers(  
2 Customer_id INT PRIMARY KEY,  
3 Customer_name VARCHAR(50),  
4 City VARCHAR(50)  
5 );  
6 CREATE TABLE orders(  
7 Order_id INT PRIMARY KEY,  
8 Order_amount DECIMAL,  
9 Order_date DATE,  
10 Customer_id INT,  
11 FOREIGN KEY Customer_id REFERENCES customers(Customer_id)  
12 );  
13 CREATE TABLE order_items(  
14 Order_item_id INT PRIMARY KEY,  
15 order_id INT,  
16 Order_item_qty INT  
17 FOREIGN KEY order_id REFERENCES orders(Order_id)  
18 );  
19 CREATE TABLE products(  
20 Product_id INT PRIMARY KEY,  
21 Order_item_id INT,  
22 Category VARCHAR(50),  
23 FOREIGN KEY Order_item_id REFERENCES order_items(Order_item_id)  
24 );
```

Lab 8: Create a Denormalized Reporting Table

Create a denormalized daily orders reporting table that contains customer_name, order_id, order_date, total_order_value, product_list (as a single string), and shipping_city.

Scenario / Input

Sample normalized data (you can assume aggregation is needed).

Daily_orders(customer_name,order_id,order_date,total_order_value,product_list_shipping_city)

```
1 CREATE TABLE Daily_orders(  
2 Order_id INT PRIMARY KEY,  
3 Customer_name VARCHAR,  
4 Order_date DATE,  
5 Total_order_value DECIMAL,  
6 Product_list TEXT,  
7 Shipping_city VARCHAR  
8  
9 );  
10 INSERT INTO Daily_orders(Order_id, Customer_name,Order_date, Total_order_value,Product_list,Shipping_city) VALUES  
11 (100, 'Alex', 12-07-2026,100.00,'Laptop,Mouse', 'Banglore');  
12 INSERT INTO Daily_orders(Order_id, Customer_name,Order_date, Total_order_value,Product_list,Shipping_city) VALUES  
13 (200, 'Alice', 12-07-2026,200.00,'Laptop,Mouse', 'Banglore');  
14  
15 SELECT  
16 *,  
17 SUM(Total_order_value) OVER () AS Total_sum  
18 FROM Daily_orders;
```

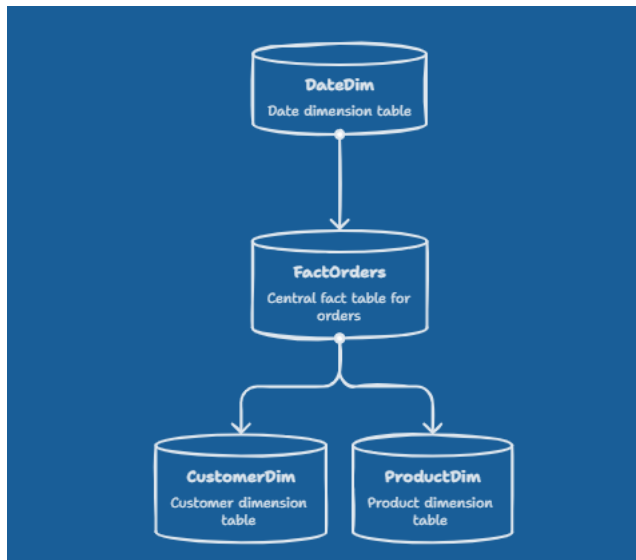
Order_id	Customer_name	Order_date	Total_order_value	Product_list	Shipping_city	Total_sum
100	Alex	-2021	100	Laptop,Mouse	Banglore	300
200	Alice	-2021	200	Laptop,Mouse	Banglore	300

Lab 9: Design a Star Schema: Fact and Dimension Tables

Design a star schema for order analytics: FactOrders and dimensions CustomerDim, ProductDim, DateDim. Define grain, keys, and 5 sample columns for each.

Scenario / Input

Analytics use cases: daily sales by product category and customer region.



```
1 CREATE TABLE CustomerDim (  
2     customer_key INT PRIMARY KEY,  
3     customer_id VARCHAR(50),  
4     customer_name VARCHAR(100),  
5     customer_region VARCHAR(50),  
6     customer_email VARCHAR(100)  
7 );  
8 CREATE TABLE ProductDim (  
9     product_key INT PRIMARY KEY,  
10    product_id VARCHAR(50),  
11    product_name VARCHAR(100),  
12    product_category VARCHAR(50),  
13    brand VARCHAR(50)  
14 );  
15 CREATE TABLE DateDim (  
16     date_key INT PRIMARY KEY,  
17     full_date DATE,  
18     day INT,  
19     month INT,  
20     year INT,  
21     day_of_week VARCHAR(10)  
22 );
```

```

CREATE TABLE FactOrders (
  order_item_id INT PRIMARY KEY,
  order_id INT,
  customer_key INT,
  product_key INT,
  date_key INT,
  quantity INT,
  total_order_value DECIMAL(10,2),
  FOREIGN KEY (customer_key) REFERENCES CustomerDim(customer_key),
  FOREIGN KEY (product_key) REFERENCES ProductDim(product_key),
  FOREIGN KEY (date_key) REFERENCES DateDim(date_key)
);

```

FactOrders Grain – one row per one order item

Means each row represents a single product purchased in a single order.

Lab 10: Map Normalized Data to Star Schema (ETL sketch)

Sketch SQL/ pseudocode to load fact table from normalized OLTP tables: orders, order_items, products, customers. Include joins and aggregations for daily fact (one row per order_item with sales_amount).

Scenario / Input

Normalized tables available in the source schema.


```

1 INSERT INTO FactOrders (order_item_id,order_id,customer_key,product_key,date_key,
2     quantity,
3     sales_amount
4 )
5 SELECT
6     oi.order_item_id,
7     oi.order_id,
8     c.customer_key,
9     p.product_key,
10    d.date_key,
11    oi.quantity,
12    oi.quantity * p.product_price AS sales_amount
13 FROM order_items oi
14 JOIN orders o
15     ON oi.order_id = o.order_id
16 JOIN customers c
17     ON o.customer_id = c.customer_id
18 JOIN products p
19     ON oi.product_id = p.product_id
20 JOIN DateDim d
21     ON o.order_date = d.full_date;
22

```

Lab 11: Model Slowly Changing Dimension (Type 2)

Design a CustomerDim that captures address history using Type 2 SCD. Show table columns and example rows demonstrating a customer moving city.

Scenario / Input

Customer 900 moved from "Oldtown" to "Newcity" on 2026-05-10.

```

1 CREATE TABLE CustomerDim(
2   customer_key INT PRIMARY KEY,
3   customer_id INT,
4   customer_name VARCHAR,
5   customer_city VARCHAR,
6   valid_from DATE,
7   valid_to DATE,
8   is_current BOOLEAN
9 );
10
11 INSERT INTO CustomerDim(customer_key,customer_id,customer_name,customer_city,valid_from,valid_to,is_current)VALUES
12 (1,900,'Alex','oldtown','2025-06-10','2026-05-10',FALSE),
13 (2,900,'Alex','Newtown','2025-06-10','2028-05-10',TRUE);
14
15 SELECT * FROM CustomerDim;

```

customer_key	customer_id	customer_name	customer_city	valid_from	valid_to	is_current
1	900	Alex	oldtown	2025-06-10	2026-05-10	0
2	900	Alex	Newtown	2025-06-10	2028-05-10	1

Lab 12: Design for Query Performance: Denormalization Trade-offs

Given slow join-heavy reports, propose three denormalization changes to speed up queries and list the trade-offs for each.

Scenario / Input

Reports frequently join orders -> customers -> addresses -> product -> category

Denormalization Option	Benefit	Trade-off
Add customer address fields to Orders	Removes customer/address joins	Duplication(same region repeated across many orders), update anomalies(customer moves → must update many rows), Large table size
Add product category to Order_Items	Removes product/category joins	Category changes require mass updates, More storage
Materialized pre-joined reporting table	Fastest reporting, no joins	ETL refresh needed, stale data risk